Université de Sherbrooke

SQL

Automatismes

SQL_07

Christina KHNAISSER *(christina.khnaisser@usherbrooke.ca)*
Luc LAVOIE *(luc.lavoie@usherbrooke.ca)*
Zeineb ZAIET *(zeineb.zaiet@usherbrooke.ca)*
(les auteurs sont cités en ordre alphabétique nominal)

—

Scriptorum/Scriptorum/SQL_07-Automatismes, version 1.0.0.a, en date du 2026-02-14

— document de travail, ne pas citer —

© 2026, CoFELI [CC BY-NC-SA 4.0]

Sommaire

…

Mise en garde

Le présent document est en cours d'élaboration ; en conséquence, il est incomplet et peut contenir des erreurs.

Historique

diffusion	resp.	description
2026-02-14	CK	Récupération de notes diverses.

© 2026, CoFELI [CC BY-NC-SA 4.0]

Table des matières

Introduction 4
1. Présentation 4
2. Les déclencheurs 4
  2.1. Syntaxe (PostgreSQL) 4
3. Les fonctions déclencheurs 5
  3.1. Syntaxe (PostgreSQL) 5
4. Exemples 5
  4.1. Exemple (1) 5
  4.2. Exemple (2) 6
Conclusion 9
Références 10
Définitions 11

© 2026, CoFELI [CC BY-NC-SA 4.0]

Introduction

Le présent document a pour but de présenter une introduction au langage de définition des automatismes qu'on retrouve dans SQL.

Dans un premier temps, nous présenterons une grammaire simplifiée, sous-ensemble propre de la grammaire de la norme ISO 9075:2016, avec, au besoin, certaines particularités de PostgreSQL.

1. Présentation

- Une fonction calcule une valeur suite à un appel
  sur la base de ses paramètres ou de l'état du calculateur, sans en changer l'état.
- Une procédure change l'état de la BD suite à un appel
  sur la base de ses paramètres et de l'état du calculateur.
- Un automatisme change l'état de la BD suite à un évènement
  sur la base de cet évènement, de ses paramètres et de l'état du calculateur.

Un calculateur est en fait une machine à états. Une base de données est un cas particulier de calculateur.

2. Les déclencheurs

2.1. Syntaxe (PostgreSQL)

```
trigger_def ::=
  CREATE [OR REPLACE] TRIGGER name
    trigger_time trigger_event ON table_name
    trigger_action

trigger_time ::=
  BEFORE | AFTER | INSTEAD OF

trigger_event ::=
  INSERT  | DELETE  | UPDATE [ OF <column name list> ]



trigger_action ::=
  [ REFERENCING { OLD | NEW } TABLE AS nom_relation_transition ]
  [ FOR EACH { ROW | STATEMENT } ]
  [ WHEN ( condition ) ]
   EXECUTE FUNCTION function_name
```

Le trigger est associé à relation (une table ou une vue) et exécute une fonction lorsque certaines opérations sont réalisées sur cette relation.

Moment du déclenchement

L'appel du trigger peut avoir lieu à différent moment :

- BEFORE : avant que l'opération ne soit tentée sur une ligne (avant la vérification des contraintes et la tentative d'INSERT, UPDATE ou DELETE) ;
- AFTER : après que l'opération soit terminée (après la vérification des contraintes et la fin de la commande INSERT, UPDATE ou DELETE) ;
- INSTEAD OF : en remplacement de l'opération.

Si le trigger est lancé avant l'événement ou en remplacement de l'événement, le trigger peut ignorer l'opération sur la ligne courante ou modifier la ligne en cours d'insertion (uniquement pour les opérations

© 2026, CoFELI [CC BY-NC-SA 4.0]

INSERT et UPDATE).

Si le trigger est activé après l'événement, toute modification, dont celles effectuées par les autres triggers, est « visible » par le trigger.

Mode d'exécution

Une action est s'exécute pour chaque ligne que l'opération modifie (FOR EACH ROW) ou une fois pour une opération donnée (FOR EACH STATEMENT)

Les triggers définis en remplacement (INSTEAD OF) doivent obligatoirement être marqués FOR EACH ROW, et ne peuvent être définis que sur des vues.

Les triggers BEFORE et AFTER portant sur des vues devront quant à eux être marqués FOR EACH STATEMENT.

Résultat intermédiare

L'option REFERENCING active la récupération des relations de transition, qui sont des ensembles de tuples incluant toutes les tuples insérées, supprimées ou modifiées par l'instruction SQL en cours.

Cette option est autorisée pour les conditions suivantes :

- le moment du déclenchement doit être AFTER.
- OLD TABLE peut seulement être indiqué pour un trigger déclenché par un UPDATE ou un DELETE, il crée une relation de transition contenant les tuples avant les mises à jour ou suppressions.
- NEW TABLE peut seulement être indiqué pour un trigger déclenché par un UPDATE ou un INSERT ; il crée une relation de transition contenant les tuples après les mises à jour ou insertions.

3. Les fonctions déclencheurs

3.1. Syntaxe (PostgreSQL)

```
trigger_function_def ::=
  CREATE [ OR REPLACE ] FUNCTION name ( [ argument [, ...] ] )
    RETURNS TRIGGER
    LANGUAGE plpgsql AS
    function_body
```

Accès à la valeur de l'objet de référence (ROW ou TABLE) :

- soit avant l'évènement (OLD)
- soit après l'évènement (NEW)

4. Exemples

4.1. Exemple (1)

Inscription des personnes étudiantes à au plus 5 activités par trimestre.

- Créer une fonction déclencheur pour vérifier le nombre d'activités par matricule et trimestre.
- Créer un déclencheur lorsqu'il y a une insertion ou une modification d'une inscription.

Définition fonction (globale)

```
create or replace function inscription_lim()
  returns trigger
  language plpgsql as
$$
```

© 2026, CoFELI [CC BY-NC-SA 4.0]

```
begin
  if not exists (select matricule
                 from inscription
                 group by matricule, trimestre
                 having count(*) > 5)
  then
    return new;
  else
    raise exception 'Le matricule est déjà inscrit à 5 activités';
  end if;
end
$$;
```

Définition du déclencheur (global)

```
create or replace trigger inscription_lim
  after insert or update on inscription
  for each statement
  execute function inscription_lim();
```

Définition fonction (spécifique)

```
create or replace function inscription_lim()
  returns trigger
  language plpgsql as
$$
begin
  if (select count(sigle)
      from inscription
      where matricule = new.matricule
        and trimestre = new.trimestre) <= 5
  then
    return new;
  else
    raise exception
      'Le matricule % est déjà inscrit à 5 activités le trimestre %.',
       new.matricule,
       new.trimestre;
  end if;
end
$$;
```

Définition du déclencheur (spécifique)

```
create or replace trigger inscription_lim
  after insert or update on inscription
  for each row
  execute function inscription_lim();
```

4.2. Exemple (2)

Admission d'une personne étudiante aux programmes d'informatique, de géomatique et d'administration.

- Créer une vue pour calculer les admissions. Si un étudiant est inscrit à au moins une activité faisant partie d'un programme, il est considéré admis à celui-ci.
- Créer une fonction déclencheur pour permettre d'insérer un étudiant et l'inscrit à des activités de son

programme.

- Créer un déclencheur pour permettre l'insertion de tuples à partir de la vue.

Définition d'une vue

```
create view Admission as
  select distinct matricule, nom, adresse, trimestre,
         case
           when estactiviteinfo(sigle) then 'Informatique'
           when substr(sigle, 1, 3) in ('GMQ') then 'Géomatique'
           when substr(sigle, 1, 3) in ('ADM') then 'Administration'
         end as programme
  from Inscription join Etudiant using (matricule);
```

Définition de la procédure

```
create or replace function admission()
  returns trigger
  language plpgsql as
$$
begin
  -- Vérifier les informations de l'étudiant
  if exists(select * from Etudiant where matricule = NEW.matricule) then
    if exists ( select *
                from Etudiant
                where matricule = NEW.matricule
                  and (nom <> NEW.nom or adresse <> NEW.adresse)) then
      raise exception 'Incohérence de nom ou d''adresse';
    else
      insert into Etudiant(matricule, nom, adresse)
        values(NEW.matricule, NEW.nom, NEW.adresse)
        on conflict do nothing;
    end if;
  else
    insert into Etudiant (matricule, nom, adresse)
      values (NEW.matricule, NEW.nom, NEW.adresse);
  end if;
  -- Inscrire la personne aux activités du programme
  if (NEW.programme = 'Informatique') then
    call inscrireinfo(new.matricule, new.nom, new.adresse, new.trimestre);
  elsif (NEW.programme = 'Géomatique') then
    insert into  Inscription(matricule, sigle, trimestre)
      select new.matricule, sigle, new.trimestre
      from Activite
      where substr(sigle, 1, 3) in ('GMQ');
  elseif (NEW.programme = 'Administration') then
    insert into  Inscription(matricule, sigle, trimestre)
      select new.matricule, sigle, new.trimestre
      from Activite
      where substr(sigle, 1, 3) in ('ADM');
  end if;
  -- Si tout va bien
  return NEW;
end
$$;
```

Définition du déclencheur

```
create or replace trigger admission
  instead of insert on Admission
```

© 2026, CoFELI [CC BY-NC-SA 4.0]

```
    for each row
    execute function admission();
```

Conclusion

© 2026, CoFELI [CC BY-NC-SA 4.0]

Références

[Elmasri2016]
Ramez ELMASRI et Shamkant B. NAVATHE;
Fundamentals of database systems;
7th Edition, Pearson, Hoboken (NJ, US), 2016;
ISBN 978-0-13-397077-7.

[Date2015a]
Chris J. DATE;
SQL and relational theory: how to write accurate SQL code;
O'Reilly Media, 2015;
ISBN 978-1-4919-4117-1.

PG

`[fr]` https://docs.postgresql.fr/18/index.html (2026-01-05)
`[en]` https://www.postgresql.org/docs/18/index.html (2026-01-05)

© 2026, CoFELI [CC BY-NC-SA 4.0]

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.
[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-10-14 03:26:33 UTC



Université de Sherbrooke

© 2026, CoFELI [CC BY-NC-SA 4.0]